Tarlac State University
College of Computer Studies
Bachelor of Science in Computer Science
Web Programming

Resource Allocation Optimization Using CPU Scheduling Techniques

A Project

Submitted to the Faculty of the College of Computer Studies

San Isidro Campus, Tarlac City

Tarlac State University

In Partial Fulfillment of the Requirements for the Course Operating Systems

Academic Year 2025-2026

Submitted By:
Manalili, Truman Jasper D.

Jo Anne Cura
Subject Instructor
Date of submission: May 15, 2026

1. INTRODUCTION

1.1 Overview of CPU Scheduling

CPU scheduling is a fundamental mechanism in modern operating systems that determines which process in the ready queue gains access to the CPU for execution. Because a CPU can execute only one process at a time in a single-core environment, the operating system must decide — often thousands of times per second — which waiting process should run next. The policy used to make this decision is called a scheduling algorithm, and its choice profoundly affects system performance, fairness, and user experience (Silberschatz et al., 2018).

Scheduling algorithms are typically evaluated across several key metrics: CPU utilization (how busy the processor is kept), throughput (how many processes complete per time unit), turnaround time (elapsed time from submission to completion), waiting time (time spent idle in the ready queue), and response time (time until the first output is produced). Different algorithms optimize for different subsets of these metrics, and no single algorithm is universally optimal (Tanenbaum & Bos, 2015).

1.2 Purpose of the Study

This document presents a practical implementation of six widely-studied CPU scheduling algorithms through an interactive desktop simulation tool built with Python 3 and the Tkinter GUI library. The simulator allows users to define custom process sets, select a scheduling policy, and immediately observe the resulting Gantt chart alongside per-process metrics and aggregate averages. By bridging theoretical algorithm descriptions with a working, visual implementation, the project reinforces conceptual understanding and provides a hands-on platform for comparing algorithm behavior.

1.3 Objectives

- Implement and demonstrate six CPU scheduling algorithms: FCFS, SJF, SRT, Round Robin, Priority (non-preemptive), and Priority + Round Robin.
- Provide an interactive GUI that accepts user-defined process parameters and renders real-time Gantt charts.
- Calculate and display Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average WT, and Average TAT for each simulation run.
- Compare algorithm behavior across representative test cases to surface practical trade-offs in fairness, efficiency, and response time.

2. SCHEDULING ALGORITHMS

The following subsections describe each of the six algorithms implemented in the simulator. For every algorithm, two sample test cases are presented, complete with input parameters, the resulting schedule (Gantt representation), and computed metrics.

2.1 First-Come, First-Served (FCFS)

FCFS is the simplest scheduling algorithm: processes are executed in the order they arrive in the ready queue. Once a process starts, it runs to completion before the CPU is assigned to the next waiting process. The implementation sorts processes by Arrival Time and appends IDLE segments whenever the CPU would otherwise be unused.

FCFS is non-preemptive — a running process cannot be interrupted by any newly arriving process, regardless of burst time or priority. This characteristic makes FCFS straightforward to implement and reason about, but it also gives rise to its most significant weakness, the convoy effect: a single long-running process can delay all shorter processes queued behind it, producing poor average waiting times when burst times are heterogeneous (Silberschatz et al., 2018). Despite this limitation, FCFS remains relevant in batch-processing environments where fairness of order is valued over minimizing mean latency, and its simplicity eliminates scheduling overhead entirely.

PID	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Table 1. FCFS Algorithm Test Case Input 1

Figure 1 illustrates the input module of the developed CPU Scheduling Simulator, wherein the First-Come, First-Served (FCFS) algorithm is selected as the scheduling policy. The interface allows users to input process-related parameters, including process ID (PID), arrival time, and burst time, which serve as the basis for scheduling computation. In this example, four processes were entered: P1 (arrival time = 0, burst time = 5), P2 (arrival time = 1, burst time = 3), P3 (arrival time = 2, burst time = 8), and P4 (arrival time = 3, burst time = 6). This figure highlights the system's user-friendly design in enabling dynamic process configuration and demonstrates how scheduling data is prepared prior to algorithm execution.

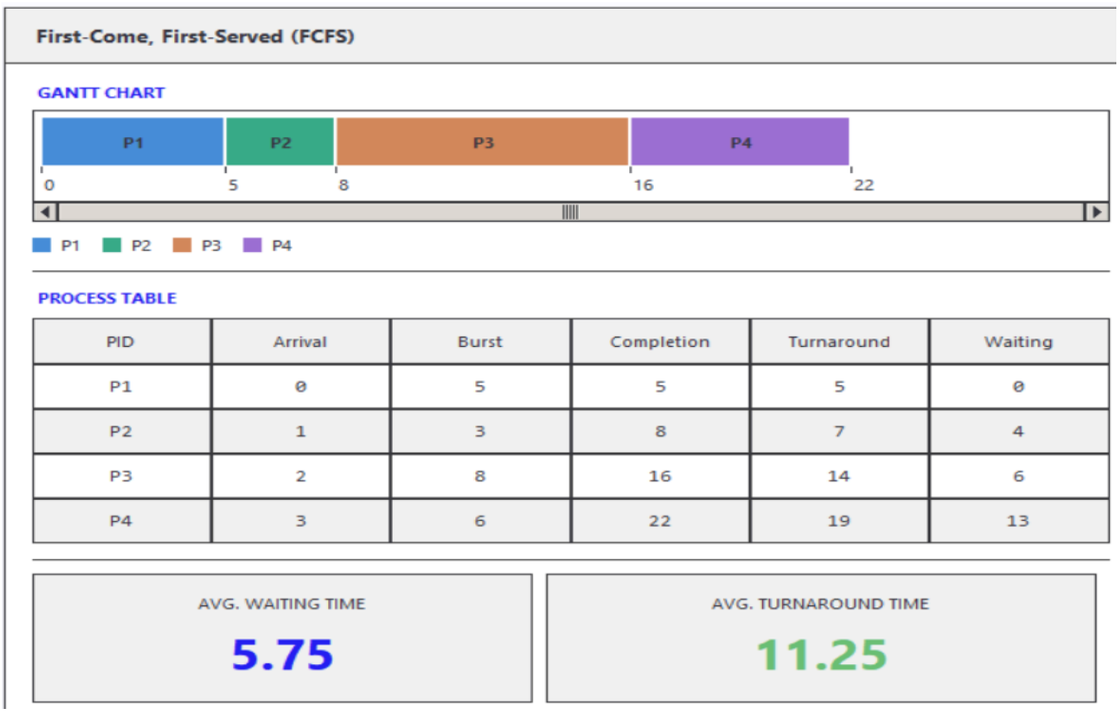


Figure 1. FCFS Algorithm Test Case 1 — Process Metrics

Figure 2 illustrates the resulting output after the FCFS algorithm processes the input data shown in Figure 1. The generated Gantt Chart visually depicts the execution sequence of processes according to their arrival order, where P1 executes from time 0–5, followed by P2 from 5–8, P3 from 8–16, and P4 from 16–22. Furthermore, the process table provides the calculated performance metrics for each process, including completion time, turnaround time, and waiting time. The simulation results reveal an average waiting time of 5.75 units and an average turnaround time of 11.25 units, confirming that the system accurately computes scheduling outcomes and effectively presents both graphical and numerical analyses for evaluating CPU scheduling performance.

PID	AT	BT
P1	0	4
P2	0	1
P3	0	2
P4	0	6

Table 2. FCFS Algorithm Test Case Input 2

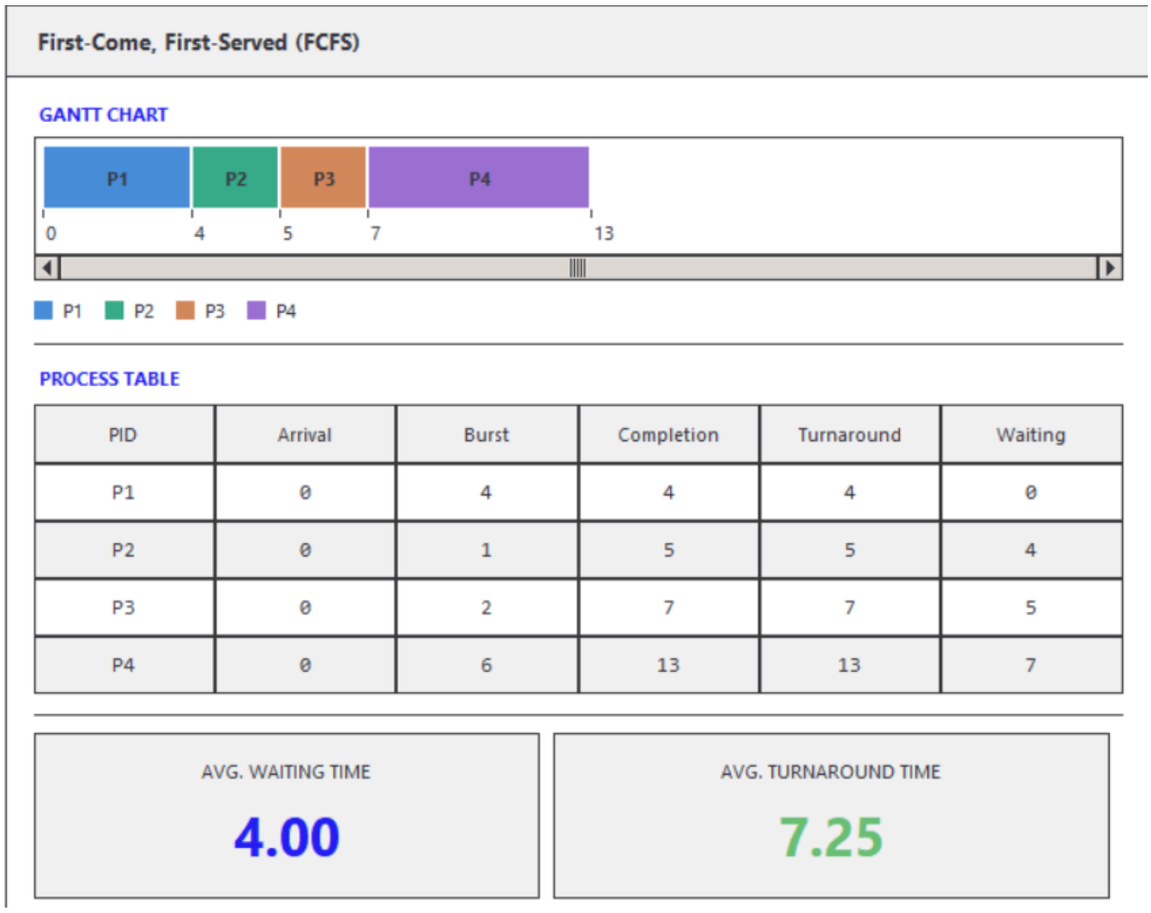


Figure 2. FCFS Algorithm Test Case 2 — Process Metrics

2.1.1 Advantages

One major advantage of FCFS is its simplicity and ease of implementation, as it follows a straightforward queue-based approach that does not require complex data structures or decision-making mechanisms. It is also considered fair in terms of arrival time since processes are served strictly in sequence, ensuring that no process is starved. Additionally, FCFS has minimal overhead because it does not involve frequent context switching or priority evaluation.

2.1.2 Limitations

However, FCFS has several notable limitations. The most significant drawback is the convoy effect, where short processes are forced to wait behind long processes, leading to increased average waiting and turnaround times. This makes the algorithm inefficient, particularly in systems with a mix of short and long jobs. Furthermore, FCFS is non-preemptive, meaning that once a process begins execution, it cannot be interrupted, which reduces responsiveness and makes it unsuitable for time-sharing or interactive systems. As a result, while FCFS is useful for simple batch systems, it performs poorly in modern computing environments that require efficiency and responsiveness.

2.2 Shortest Job First — Non-preemptive

SJF selects the process with the shortest Burst Time from among all currently available processes. Once selected, the process runs to completion without interruption. When multiple processes have equal burst times, arrival order serves as a tiebreaker. SJF is provably optimal in minimizing average waiting time for a given static batch of processes when burst times are known in advance (Silberschatz et al., 2018).

The principal limitation of SJF in practice is that burst time is rarely knowable before execution. Real systems approximate SJF through exponential averaging of past CPU bursts, introducing estimation error and implementation complexity. Furthermore, in dynamic systems where short jobs arrive continuously, long processes can experience indefinite starvation — a problem addressable only through aging mechanisms that artificially inflate the effective priority of waiting processes over time (Tanenbaum & Bos, 2015).

PID	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Table 3. SJF Algorithm Test Case Input 1

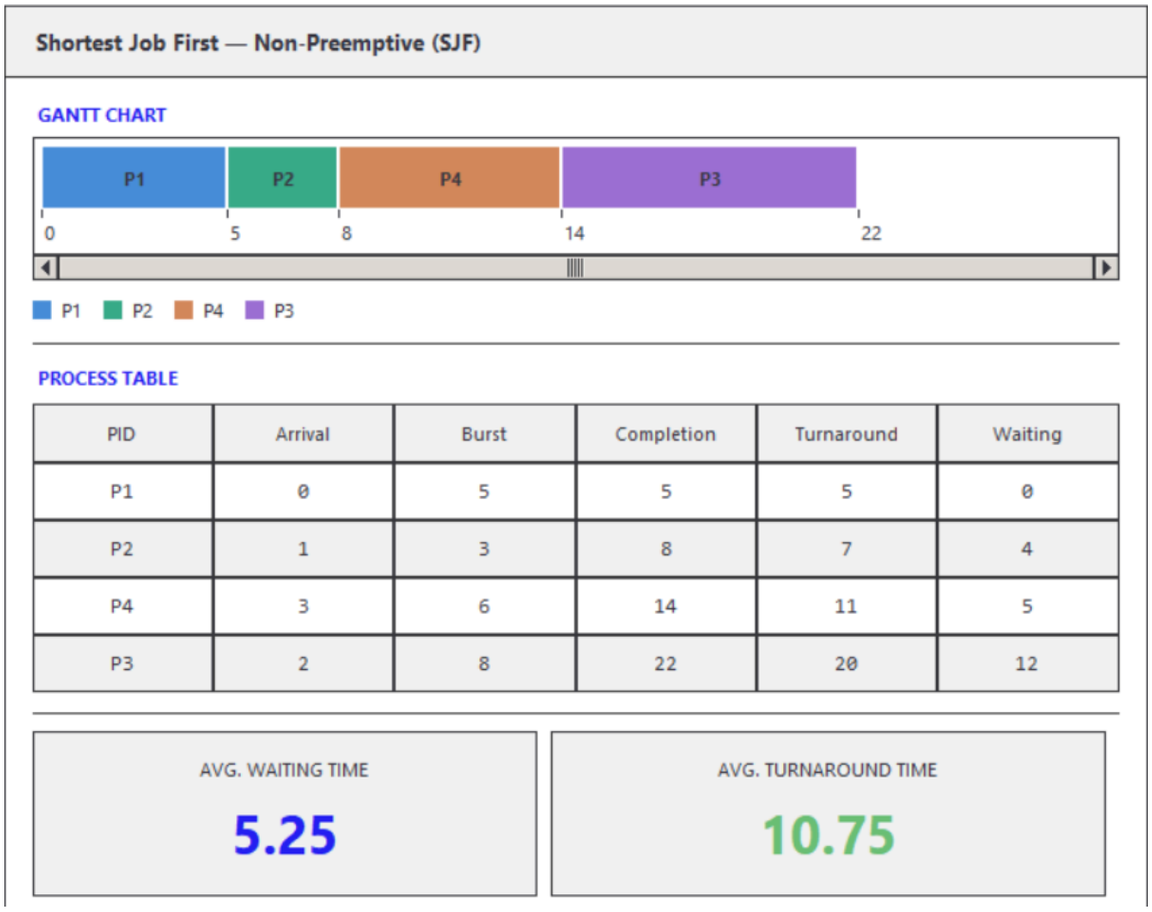
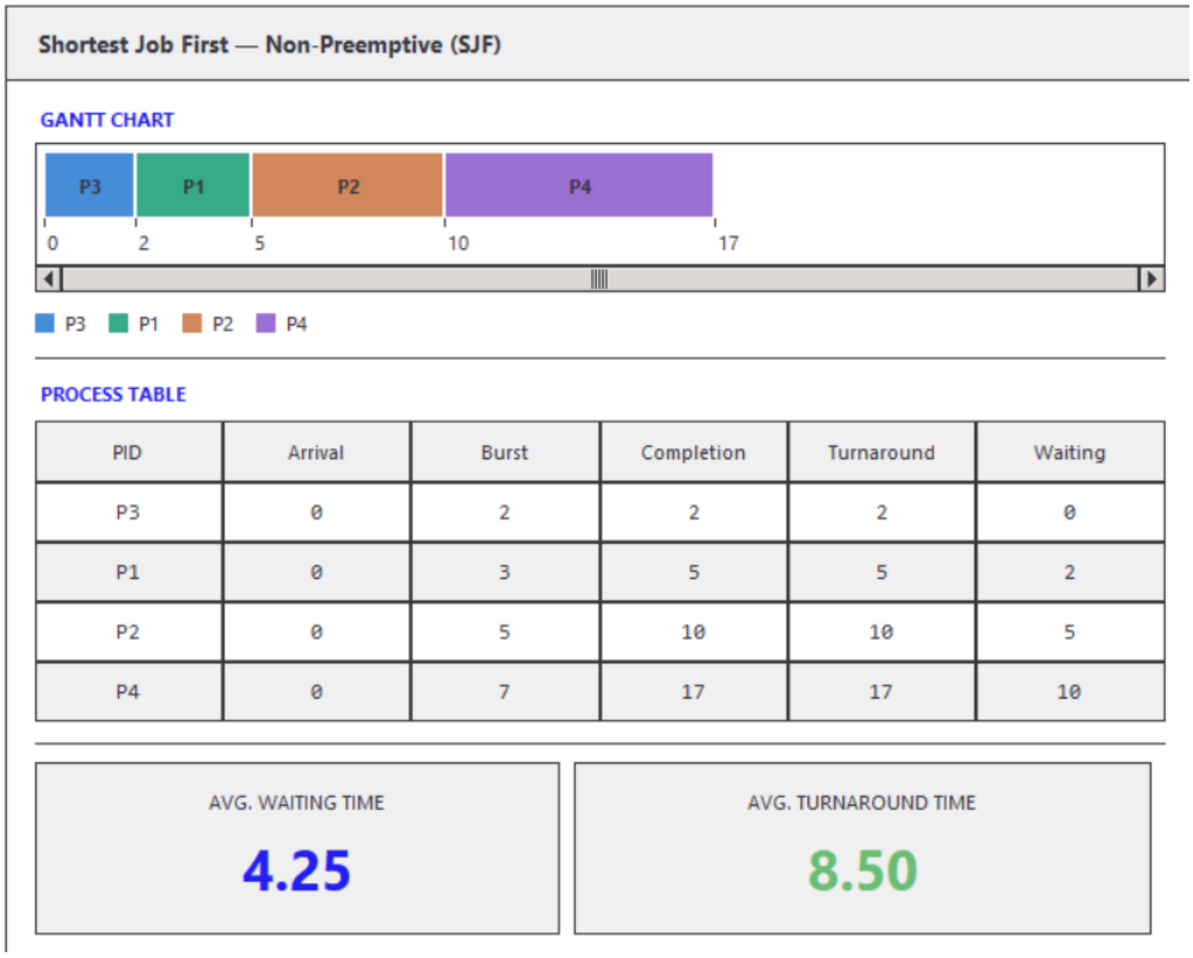


Figure 3. SJF Algorithm Test Case 1 — Process Metrics

PID	AT	BT
P1	0	3
P2	0	5
P3	0	2
P4	0	7

Table 4. SJF Algorithm Test Case Input 2



PID	Arrival	Burst	Completion	Turnaround	Waiting
P3	0	2	2	2	0
P1	0	3	5	5	2
P2	0	5	10	10	5
P4	0	7	17	17	10

Figure 4. SJF Algorithm Test Case 2 — Process Metrics

When all processes arrive simultaneously at time 0 in Test Case 2, the CPU's ready queue is fully populated from the very first instant, allowing Shortest Job First to make scheduling decisions based purely on burst time without any interference from staggered arrivals. The resulting execution order — P3, P1, P2, P4, reflecting ascending burst times of 2, 3, 5, and 7 — is not coincidental but mathematically optimal. Because every process shares the same arrival time, each process's waiting time equals the cumulative sum of all burst times that preceded it, meaning any process scheduled earlier directly inflates the waiting time of every process that follows. By always selecting the shortest remaining job first, SJF minimizes this accumulation: P3 contributes zero wait, P1 waits only 2 units, P2 waits 5, and P4 waits 10, yielding an average waiting time of 4.25 units — the lowest achievable for this process set. This can be verified through the exchange argument: if any two adjacent processes were swapped such that a longer burst ran before a shorter one, every subsequent process would inherit the difference in burst times as additional unnecessary waiting, increasing the overall average. Test Case 2 thus serves as the definitive demonstration of SJF's optimality, as the absence of varying arrival times eliminates the scheduling tradeoffs that complicate real-world scenarios and reveals the algorithm's core principle in its purest form.

2.2.1 Advantages

Shortest Job First (SJF) scheduling offers several notable advantages in process management. It minimizes the average waiting time compared to other non-preemptive scheduling algorithms by prioritizing processes with the shortest burst times, thereby improving overall system efficiency. This approach also provides good throughput, particularly in CPU-bound batch workloads, as shorter tasks are completed quickly, allowing more processes to be handled within a given period. Additionally, SJF is relatively straightforward to implement in environments where process burst times are known in advance, making it effective for predictable workloads.

2.2.2 Limitations

Despite these benefits, SJF has several limitations that affect its practicality in real-world systems. One major drawback is that burst time is rarely known beforehand, making accurate scheduling difficult in dynamic computing environments. Furthermore, the algorithm may lead to starvation, as longer processes can experience indefinite delays when shorter jobs continuously arrive. Lastly, SJF is less suitable for interactive systems because it lacks responsiveness to short or high-priority requests that arrive later, which may negatively impact user experience and system performance.

2.3 Shortest Remaining Time — Preemptive

SRT is the preemptive extension of SJF. At every clock tick, the scheduler compares the remaining burst time of the currently executing process against all processes that have newly arrived. If any waiting process has a shorter remaining time than the process currently holding the CPU, a preemption occurs and the shorter job takes over immediately. The simulator implements SRT through a tick-by-tick loop and consolidates consecutive Gantt segments belonging to the same process.

SRT achieves optimal average waiting time among all scheduling policies, a result that follows directly from the optimality of SJF extended to the preemptive case (Arpaci-Dusseau & Arpaci-Dusseau, 2023). However, this theoretical optimality comes at practical cost: frequent preemptions impose context-switch overhead, and the continuous need to know or estimate remaining burst time limits SRT's direct applicability to systems without reliable burst prediction. Long-running processes face particularly acute starvation risk when short jobs arrive continuously.

PID	AT	BT
P1	0	5
P2	1	3

P3	2	8
P4	3	6

Table 5. SRT Algorithm Test Case Input 1

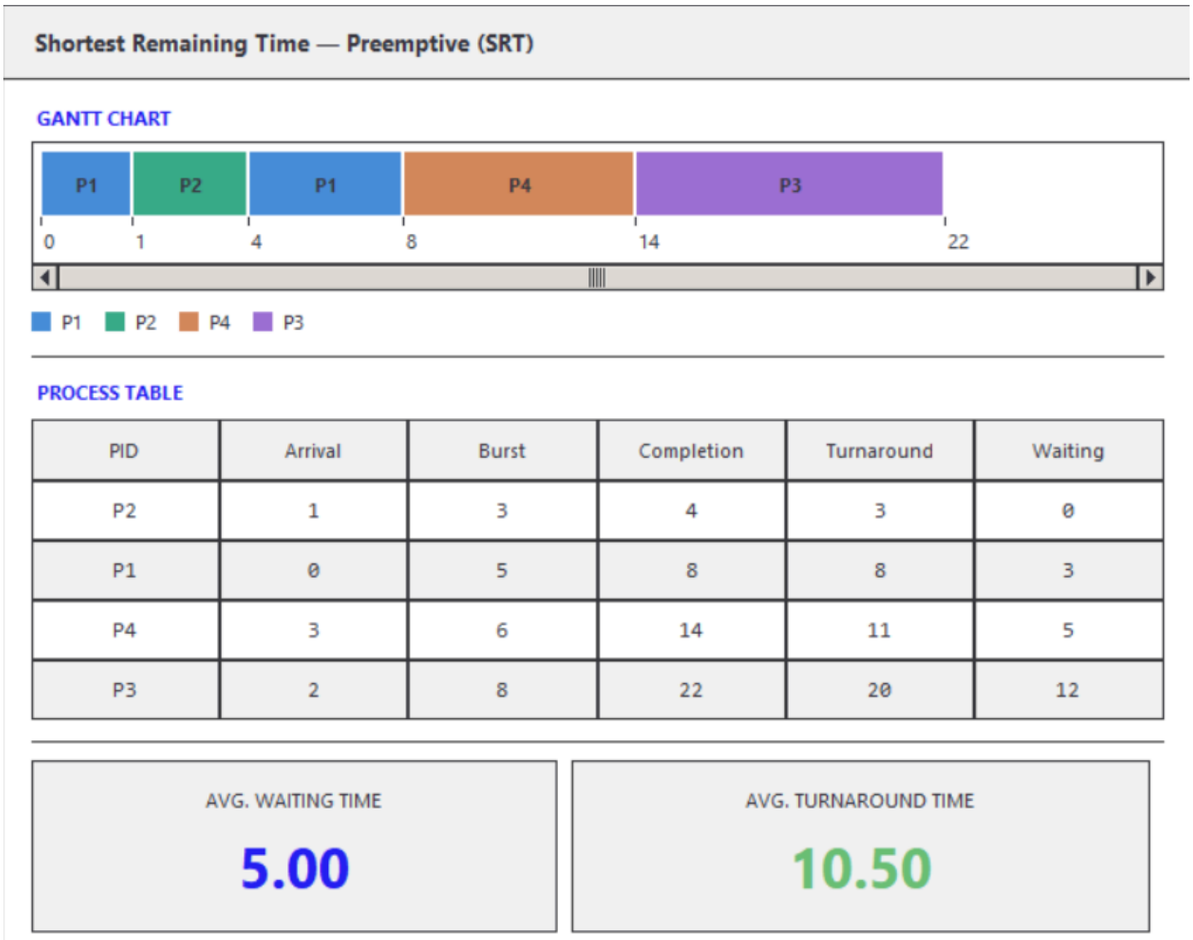
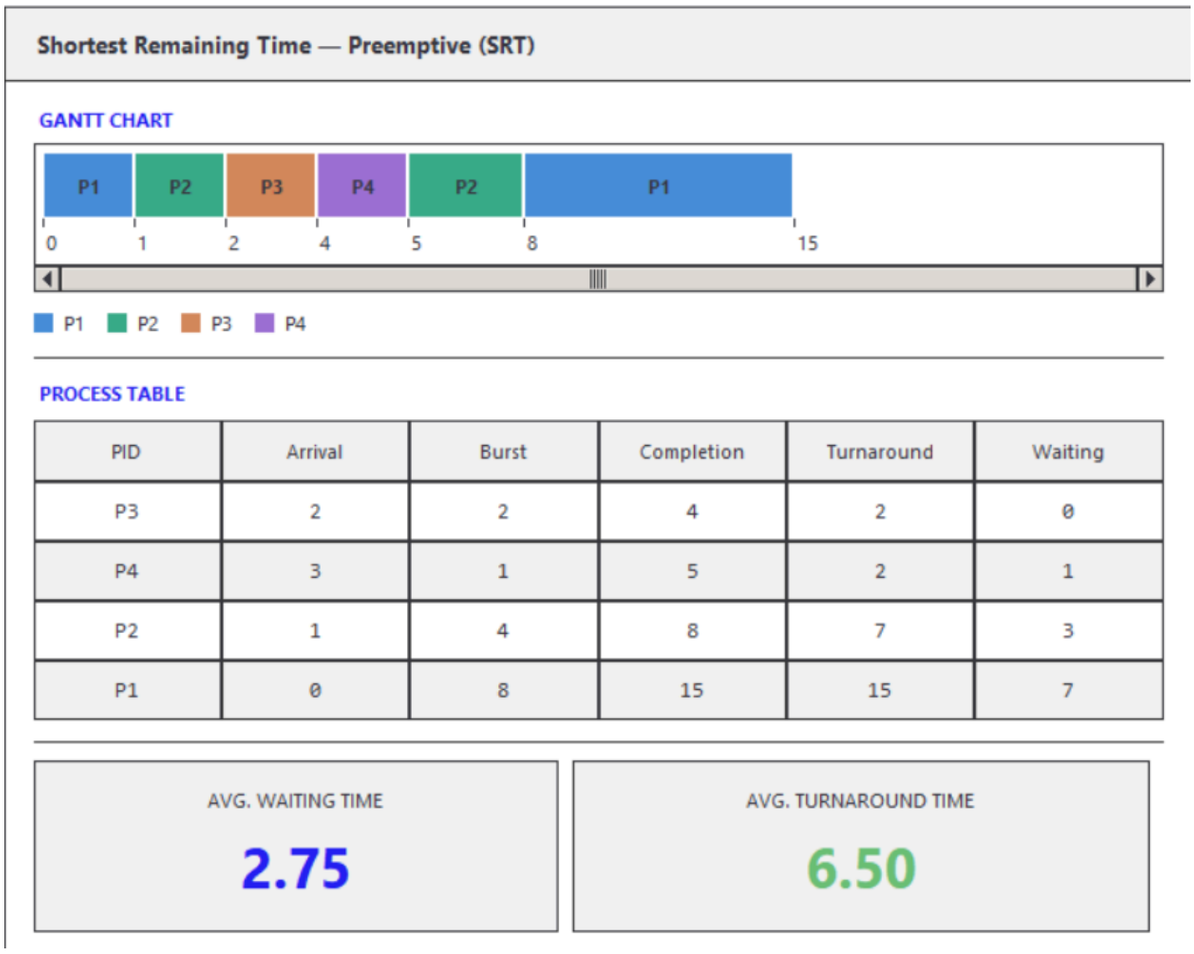


Figure 5. SRT Algorithm Test Case 1 — Process Metrics

PID	AT	BT
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Table 6. SRT Algorithm Test Case Input 2



PID	Arrival	Burst	Completion	Turnaround	Waiting
P3	2	2	4	2	0
P4	3	1	5	2	1
P2	1	4	8	7	3
P1	0	8	15	15	7

Figure 6. SRT Algorithm Test Case 2 — Process Metrics

Test Case 2 illustrates the characteristic preemption cascade of SRT: each newly arriving process (P2 through P4) carries a shorter remaining time than the process currently executing, producing a sequence of preemptions before the shorter jobs complete and longer processes resume. This cascade minimizes average waiting time (2.75) but at the cost of significant context-switch activity.

2.3.1 Advantages

Shortest Remaining Time (SRT), the preemptive version of Shortest Job First scheduling, provides several advantages in process management and system performance. It achieves the optimal average waiting time among scheduling policies by continuously prioritizing processes with the shortest remaining execution time. This characteristic makes the algorithm highly responsive to short processes and newly arriving tasks, as they can immediately preempt longer-running processes when necessary. Consequently, SRT is particularly well-suited for interactive systems where quick response time is essential to maintaining efficient user interaction and overall system responsiveness.

2.3.2 Limitations

The algorithm incurs higher context-switch overhead due to frequent preemption, which may reduce system performance when switching occurs excessively. Additionally, it requires accurate knowledge or estimation of the remaining burst time of processes, which is often difficult to determine in real-world operating systems. Furthermore, long processes may experience indefinite starvation in highly dynamic environments where shorter tasks continuously enter the ready queue, resulting in delayed execution for longer jobs.

2.4 Round Robin (RR)

Round Robin assigns each process a fixed time slice called the time quantum (Q). Processes are served in circular order: when a process exhausts its quantum without finishing, it is placed at the back of the ready queue and the next process runs. Newly arriving processes join the queue before the current process is re-queued, matching the behavior of most real operating system implementations (Tanenbaum & Bos, 2015). The test cases below use Q = 2.

Round Robin's fundamental characteristic is fairness: every process is guaranteed CPU access within a bounded period proportional to the quantum. This property makes RR well-suited to interactive and time-sharing systems. However, algorithm performance is highly sensitive to quantum selection. A quantum that is too small causes excessive context-switch overhead, as each switch consumes CPU cycles without advancing useful work; a quantum that is too large causes RR to degrade toward FCFS behavior. Optimal quantum selection typically requires knowledge of the workload's burst-time distribution — information that is often unavailable in advance (Love, 2010).

PID	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Table 7. Round Robin Algorithm Test Case Input 1

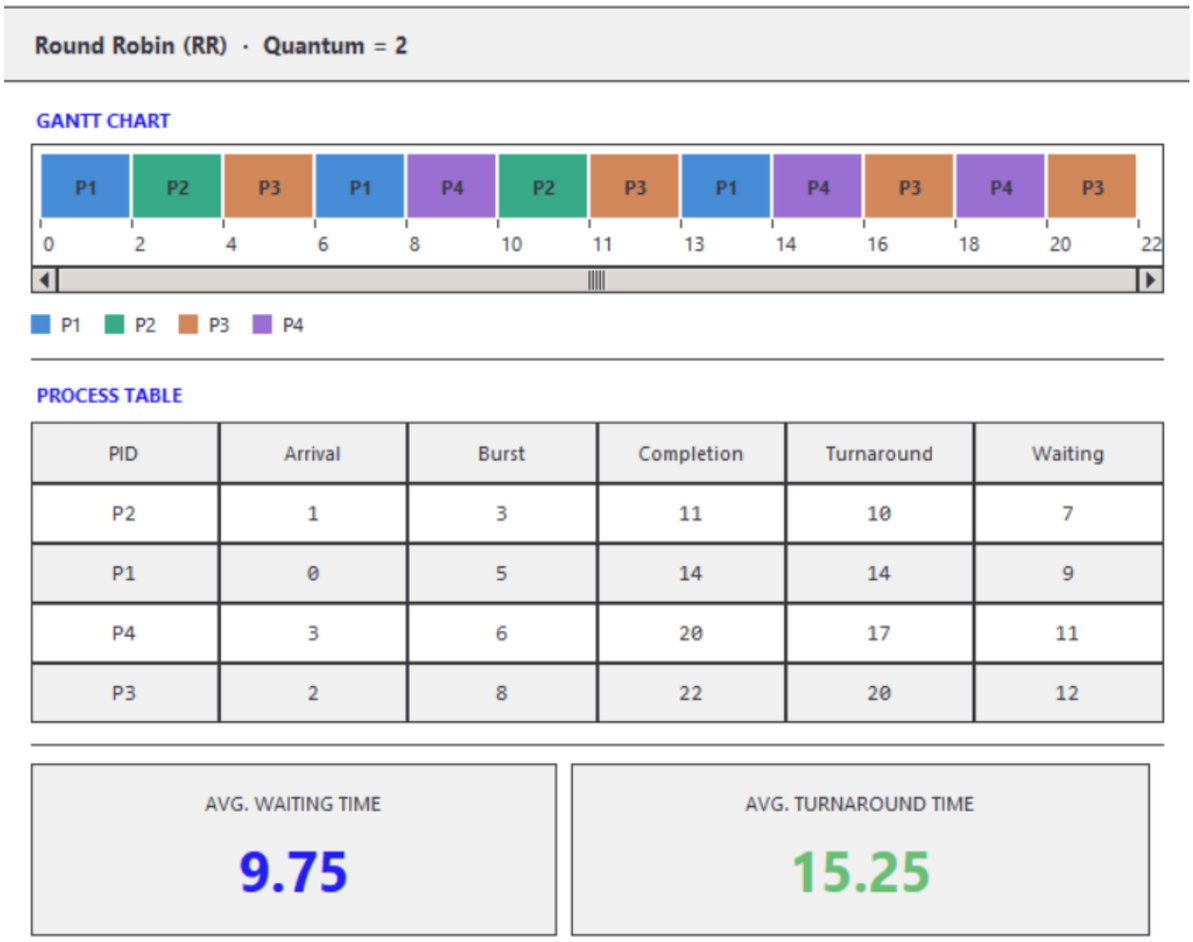


Figure 7. Round Robin Algorithm Test Case 1 — Process Metrics

PID	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Table 8. Round Robin Algorithm Test Case Input 2

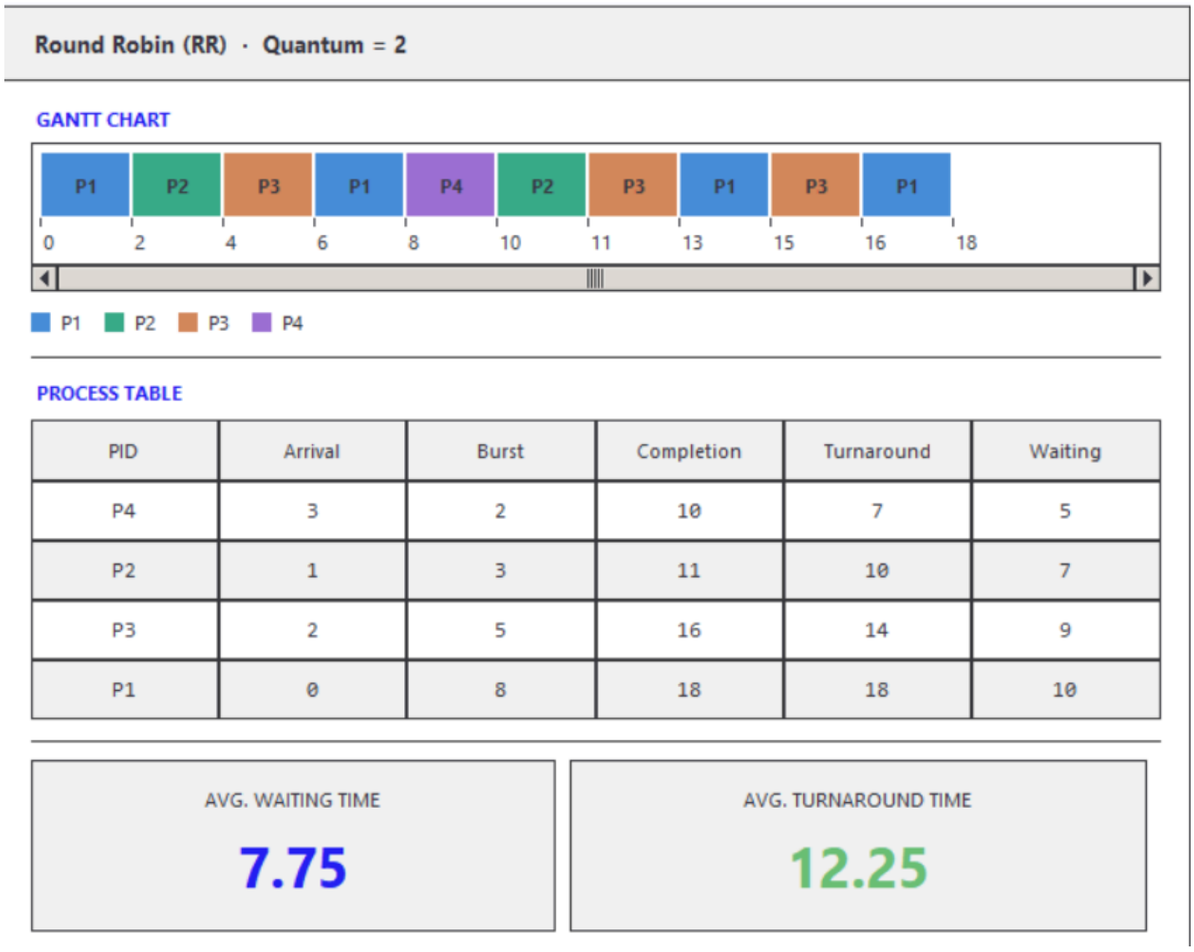


Figure 8. Round Robin Algorithm Test Case 2 — Process Metrics

The higher average waiting times observed for RR compared to SJF or SRT reflect the cost of equitable distribution: no single process receives early preferential treatment, so each process accumulates waiting time proportionally. This trade-off is acceptable — and often desirable — in interactive workloads where response-time guarantees matter more than minimizing mean waiting time across the batch.

2.4.1 Advantages

Round Robin (RR) scheduling offers several advantages in managing processes within time-sharing systems. It ensures fair allocation of CPU time by allowing each process to execute for a fixed time quantum in a cyclic order, preventing any single process from monopolizing system resources. This approach provides good response time for interactive workloads, as processes are given regular opportunities to execute, which enhances user experience in multitasking environments. Moreover, Round Robin eliminates starvation since every process is guaranteed CPU access within a bounded period, ensuring fairness and predictability in process execution.

2.4.2 Limitations

Its performance is highly dependent on the selection of the time quantum, as an excessively large quantum may cause the algorithm to behave similarly to First-Come, First-Served scheduling, while an extremely small quantum can result in excessive context switching. Frequent context switches introduce additional overhead that may reduce overall system efficiency. Furthermore, the average waiting time in Round Robin can be higher than that of Shortest Job First scheduling, particularly in workloads with highly varied burst times, making it less optimal for minimizing waiting time.

2.5 Priority Scheduling — Non-preemptive

Priority scheduling assigns each process a numeric priority. When the CPU becomes free, the ready process with the lowest priority number — indicating the highest urgency — is selected for execution. Ties are broken by arrival time and then by process ID. The algorithm is structurally analogous to SJF, but uses explicit priority values rather than burst time as the selection key, allowing external semantics (e.g., system vs. user processes, deadline criticality) to govern scheduling decisions.

Its primary risk is starvation: in a workload with continuously arriving high-priority processes, low-priority processes may never gain CPU access. The canonical mitigation is aging — a mechanism that incrementally increases the effective priority of waiting processes over time, guaranteeing eventual execution. Without aging, static priority scheduling is inappropriate for general-purpose workloads (Stallings, 2018). Priority inversion, where a high-priority process is indirectly blocked by a lower-priority one through shared resources, is a related hazard addressed through protocols such as priority inheritance.

PID	AT	BT	PRIORITY
P1	0	5	1
P2	1	3	2
P3	2	8	3
P4	3	6	2

Table 9. Priority Scheduling Algorithm Test Case Input 1

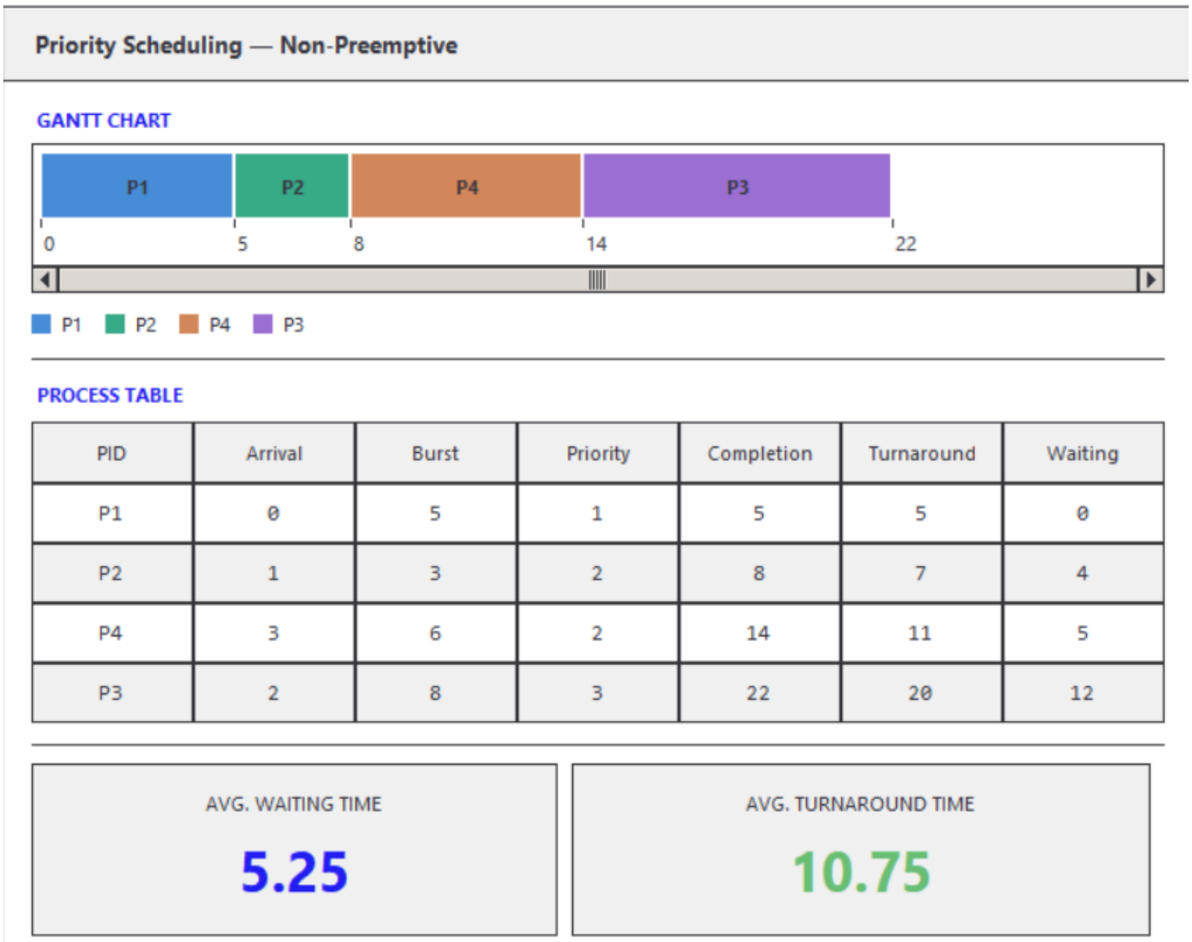
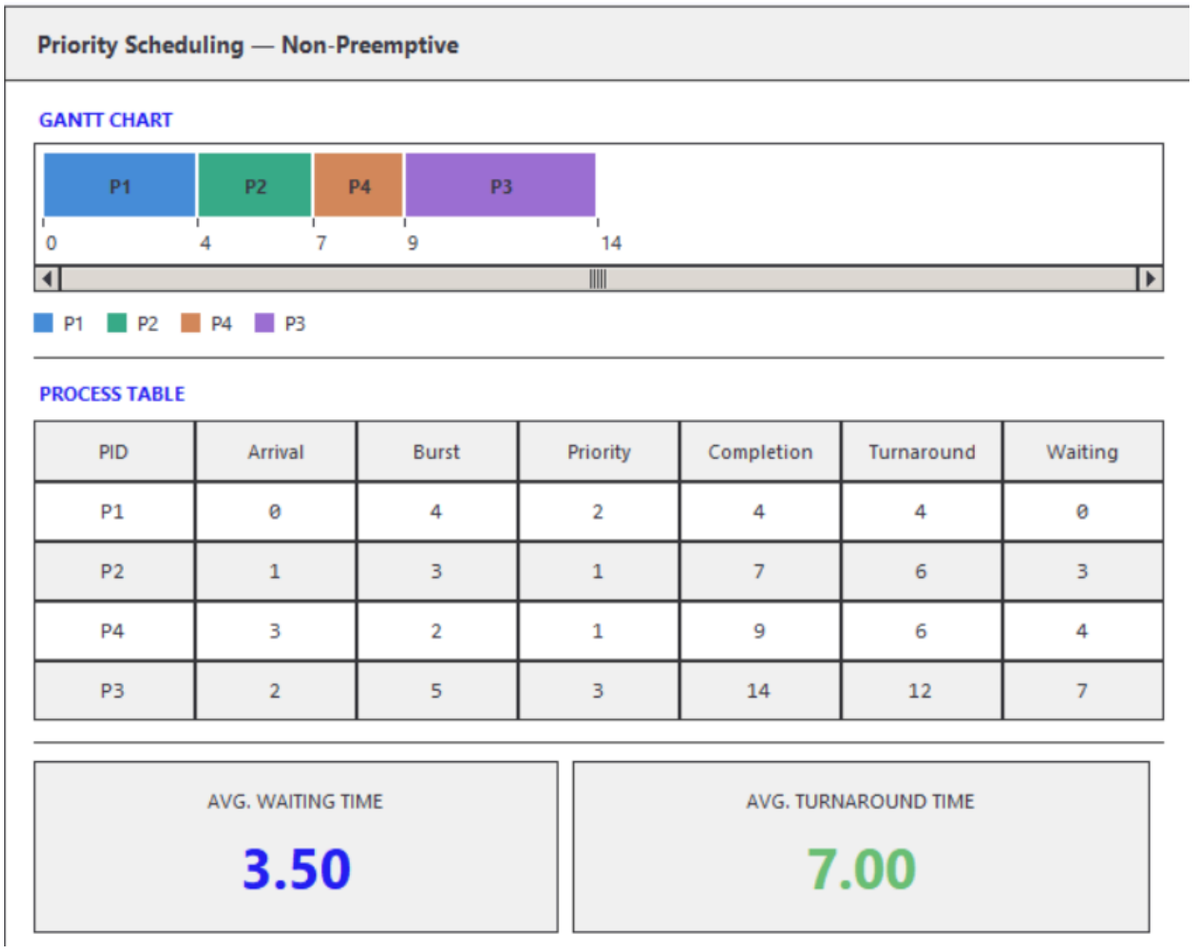


Figure 9. Priority Scheduling Algorithm Test Case 1 — Process Metrics

PID	AT	BT	PRIORITY
P1	0	4	2
P2	1	3	1
P3	2	2	1
P4	3	5	3

Table 10. Priority Scheduling Algorithm Test Case Input 2



PID	Arrival	Burst	Priority	Completion	Turnaround	Waiting
P1	0	4	2	4	4	0
P2	1	3	1	7	6	3
P4	3	2	1	9	6	4
P3	2	5	3	14	12	7

Figure 10. Priority Scheduling Algorithm Test Case 1 — Process Metrics

Test Case 2 demonstrates that non-preemptive priority scheduling allows an already-running lower-priority process (P1, priority 2) to complete before P2 (priority 1, arriving at time 1) can take over. This underscores the practical distinction between non-preemptive and preemptive priority variants: in the former, arriving high-priority processes must wait for the current process to finish regardless of urgency.

2.5.1 Advantages

It enables differentiated service levels by allowing critical system processes to be prioritized over less important user processes, ensuring that high-priority tasks receive immediate attention. The algorithm is also relatively simple to implement, as priority values can be assigned based on external deadlines, task importance, or system requirements. Additionally, Priority Scheduling is effective in environments where workload characteristics are known at the time of submission, allowing the system to allocate resources according to predefined priorities.

2.5.2 Limitations

One major issue is the potential starvation of low-priority processes, as they may experience indefinite delays when higher-priority tasks continuously enter the system. This problem often requires mitigation techniques such as aging, which gradually increases the

priority of waiting processes over time. Furthermore, priority inversion may occur when lower-priority processes hold resources needed by higher-priority tasks, unless mechanisms such as priority inheritance are implemented. Lastly, static priority assignments may fail to adapt to dynamic changes in process importance, reducing the algorithm's effectiveness in highly variable computing environments.

2.6 Priority Scheduling + Round Robin

The Priority with Round Robin hybrid algorithm first partitions processes into groups by priority level, then applies Round Robin scheduling independently within each group. All processes at the highest priority level complete before any lower-priority group begins execution. Within a group, standard RR quantum rules apply, balancing fairness among processes that share the same priority tier. The implementation processes priority levels in ascending numeric order (lowest number = highest priority), consistent with multilevel queue scheduler designs found in production operating systems (Stallings, 2018). Preemption operates within a priority level via RR quanta but is non-preemptive across priority levels.

This hybrid approach captures the principal benefit of each constituent policy: strict inter-level ordering from priority scheduling ensures that high-urgency work is not delayed by lower-priority processes, while intra-level Round Robin prevents any single process within a priority tier from monopolizing the CPU. Real-world analogues include Windows thread priority classes and Linux's multilevel feedback queue, which dynamically adjust process priority based on observed CPU-boundedness and I/O behavior (Love, 2010).

PID	AT	BT	PRIORITY
P1	0	5	1
P2	1	3	2
P3	2	8	3
P4	3	6	1

Table 11. Priority Scheduling + Round Robin Algorithm Test Case Input 1

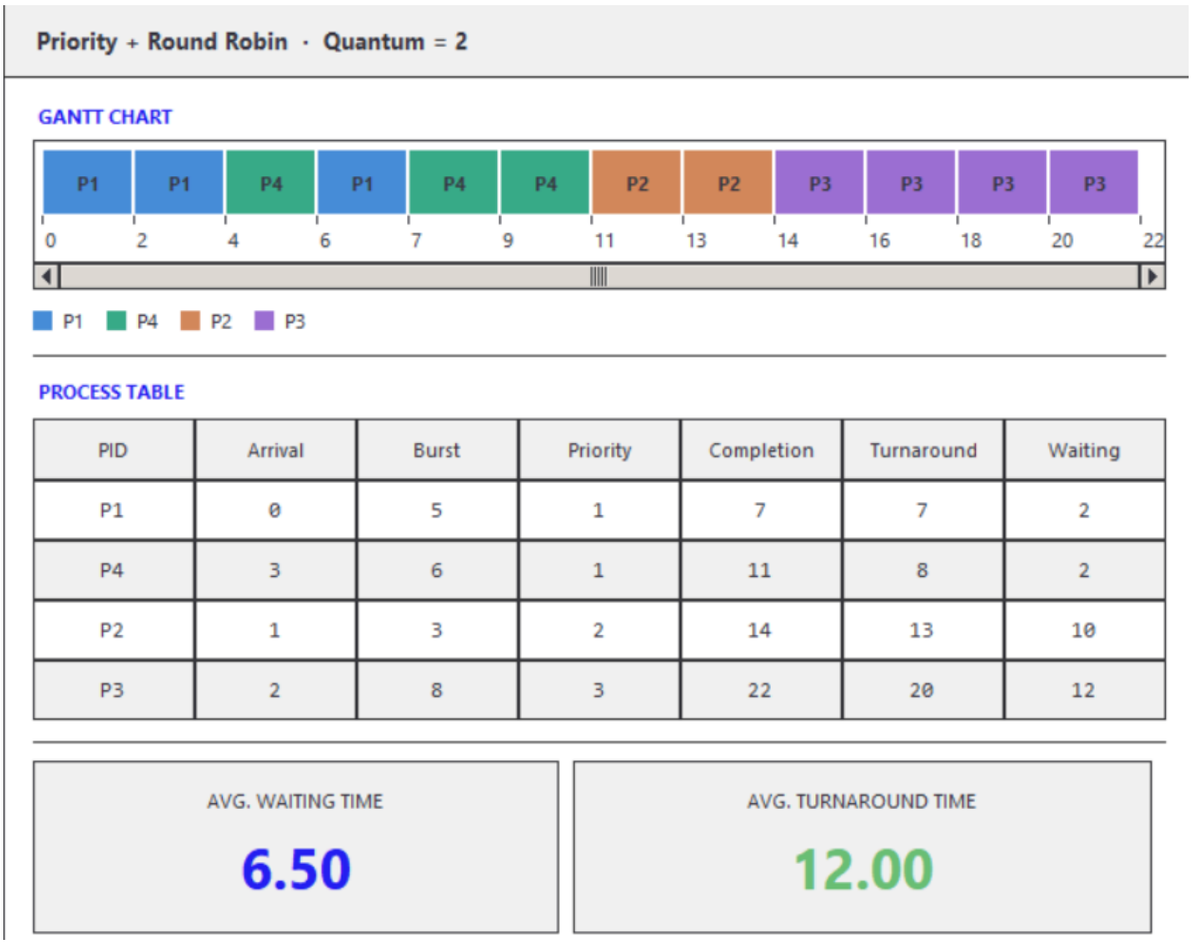
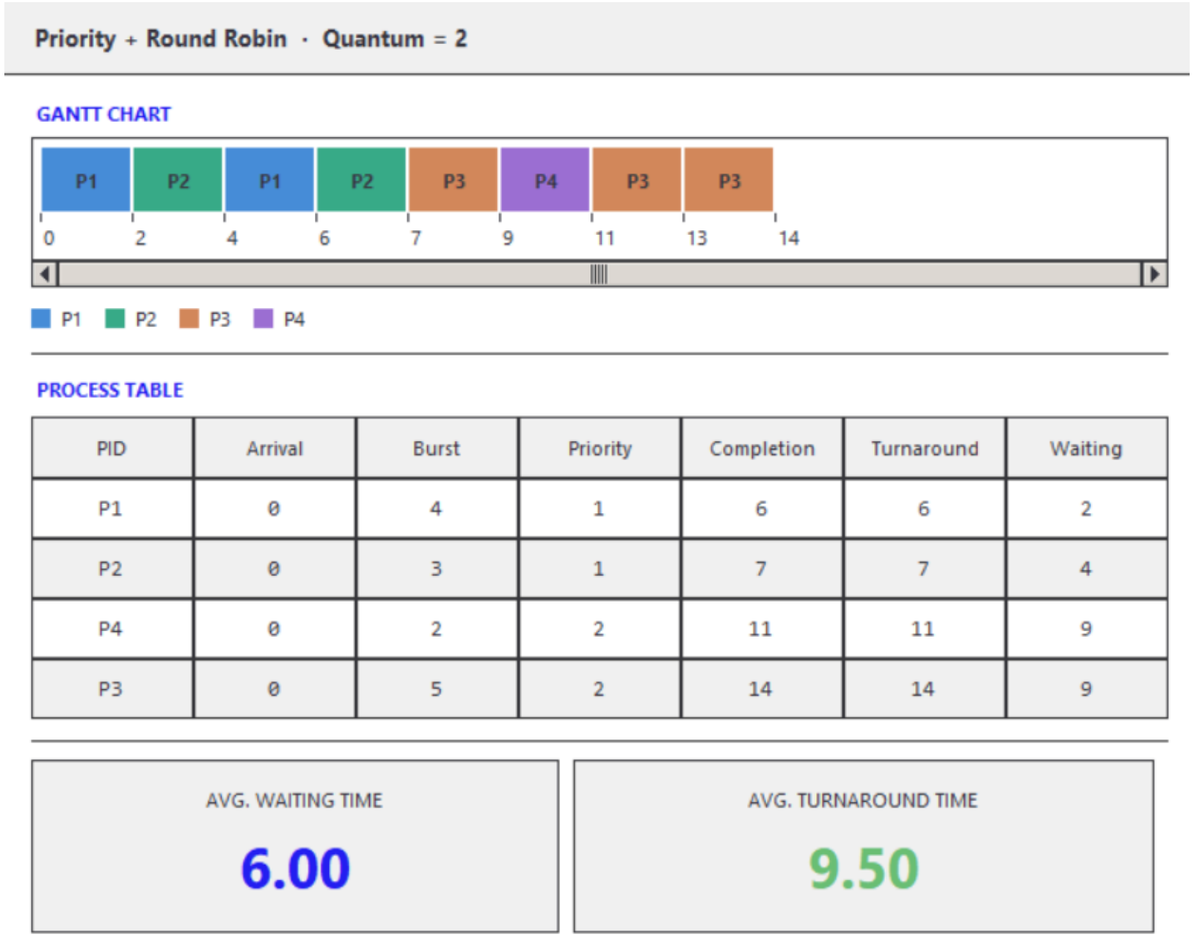


Figure 11. Priority Scheduling + Round Robin Algorithm Test Case 1 — Process Metrics

PID	AT	BT	PRIORITY
P1	0	4	2
P2	1	3	1
P3	2	2	1
P4	3	5	3

Table 12. Priority Scheduling + Round Robin Algorithm Test Case Input 2



PID	Arrival	Burst	Priority	Completion	Turnaround	Waiting
P1	0	4	1	6	6	2
P2	0	3	1	7	7	4
P4	0	2	2	11	11	9
P3	0	5	2	14	14	9

Figure 12. Priority Scheduling + Round Robin Algorithm Test Case 2 — Process Metrics

2.6.1 Advantages

Priority Round Robin Scheduling combines the strengths of both Priority Scheduling and Round Robin algorithms, making it suitable for handling multi-class workloads in modern operating systems. One of its primary advantages is that it ensures fairness among processes within the same priority level by allocating CPU time using a round-robin approach, while still maintaining strict execution order across different priority groups. This prevents lower-priority processes from interfering with high-priority tasks, ensuring that critical processes are completed first. Additionally, this hybrid approach is considered more realistic than pure Priority Scheduling because it better accommodates environments where multiple categories of workloads with varying priorities must be managed simultaneously.

2.6.2 Limitations

The integration of two scheduling mechanisms increases implementation complexity compared to using either Priority Scheduling or Round Robin alone, requiring more sophisticated process management. Furthermore, lower-priority groups may still experience starvation if the higher-priority queue contains a large number of processes that continuously

demand CPU resources. Another challenge lies in selecting an optimal time quantum, as determining appropriate quantum values becomes more difficult when multiple priority levels are involved, potentially affecting system efficiency and responsiveness.

2.7 Comparative Analysis

The table below summarizes the key characteristics of all six implemented algorithms, enabling direct comparison across the dimensions most relevant to scheduling policy selection.

Algorithm	Preemptive	Avg WT (Case 1)	Starvation Risk	Best For
FCFS	No	5.75	No	Batch / simple jobs
SJF	No	5.25	Yes	Known-burst batch
SRT	Yes	2.75	Yes	Interactive systems
Round Robin	Yes	9.75	No	Time-sharing / fairness
Priority NP	No	5.25	Yes	Differentiated classes
Priority + RR	Yes	6.50	Partial	Multi-class interactive

Figure 13. Comparison of CPU Scheduling Algorithms Based on Performance and Characteristics

The figure illustrates a comparative evaluation of widely used CPU scheduling algorithms, namely First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time (SRT), Round Robin (RR), Non-Preemptive Priority, and a hybrid Priority with Round Robin approach. The comparison is based on key performance indicators, including preemptive behavior, average waiting time (Case 1), susceptibility to starvation, and suitability for specific system environments. The results demonstrate the inherent trade-offs among the algorithms, where non-preemptive methods such as FCFS and SJF offer simplicity but may suffer from inefficiencies like increased waiting time or starvation. In contrast, preemptive algorithms such as SRT and Round Robin provide improved responsiveness and fairness, making them more appropriate for interactive and time-sharing systems, albeit with higher overhead and average waiting times in certain scenarios.

3. SUMMARY

3.1 Key Insights

There is no single scheduling algorithm that is universally optimal across all system conditions. Instead, each policy involves explicit trade-offs among key performance metrics such as average waiting time, fairness, response time, and implementation complexity. Preemptive scheduling approaches, including Shortest Remaining Time and preemptive priority or Round Robin variants, typically improve system responsiveness, particularly for interactive workloads; however, this improvement comes at the expense of increased context-switching overhead and greater implementation complexity. In the case of Round Robin scheduling, performance is highly sensitive to the choice of time quantum: a very small quantum leads to excessive context-switching overhead, while a very large quantum causes the algorithm to approximate First-Come, First-Served behavior, reducing its responsiveness advantages. Priority-based scheduling, when implemented without an aging mechanism, introduces the risk of starvation, where lower-priority processes may be indefinitely delayed under sustained high-priority load.

Hybrid approaches, such as the combination of Priority Scheduling with Round Robin, illustrate how modern operating systems manage diverse workload requirements by layering multiple scheduling strategies. This design philosophy is reflected in contemporary systems such as Linux and Windows, where scheduling frameworks integrate priority handling with fairness-oriented mechanisms to balance efficiency and responsiveness across different classes of processes.

3.2 Performance Difference

In test cases featuring wide burst-time variance, SRT consistently achieved the lowest average waiting time, confirming its theoretical optimality. FCFS suffered severe convoy effects when long processes arrived first. Round Robin provided the most equitable CPU distribution but at a higher average waiting time. Priority scheduling was effective when priorities correctly reflected urgency, but highlighted how static priorities can be suboptimal in dynamic environments.

3.3 Practical Relevance

CPU scheduling is not merely an academic exercise. Modern operating systems deploy sophisticated multilevel feedback queue schedulers that dynamically adjust priorities based on process behavior — effectively combining insights from all six algorithms studied here. Understanding the primitives (FCFS, SJF, RR, Priority) is essential for diagnosing performance problems, tuning system parameters, and reasoning about the fairness and latency guarantees offered by production schedulers (Love, 2010).

3.4 Challenges Encountered

- Correctly implementing SRT tick-by-tick preemption while merging consecutive same-process Gantt bars required careful state management to avoid off-by-one errors.
- Round Robin's ready-queue ordering is sensitive to the exact moment newly arrived processes are admitted, requiring the implementation to enqueue arrivals before re-queuing the current process.
- The Priority + Round Robin hybrid required a clear decision on how to handle simultaneous cross-group process arrivals during a lower-priority group's RR cycle.
- Designing a Tkinter GUI that remains responsive and scrollable with an arbitrarily long Gantt chart demanded a horizontally scrollable canvas with dynamic scroll-region updates.

3.5 Conclusion

This case study has implemented and analyzed six CPU scheduling algorithms through an interactive Python simulation, providing both empirical performance data and practical implementation experience. The results confirm central theoretical predictions from the operating systems literature: SRT achieves optimal average waiting time at the cost of frequent preemptions; FCFS is susceptible to convoy effects under heterogeneous burst times; Round Robin provides fairness and bounded response time at the expense of higher mean waiting time; and priority-based schemes enable differentiated service while risking starvation without aging.

Beyond validating theory, the simulation experience surfaces the gap between algorithmic description and working implementation — a gap that encompasses preemption timing, queue ordering decisions, Gantt chart merging, and GUI responsiveness. These practical dimensions are as important to understanding real schedulers as the theoretical performance bounds they approximate.

Future work could extend this study in several directions: implementing exponential averaging for burst-time prediction in SJF and SRT, adding aging to the priority scheduler, introducing multicore simulation, or comparing simulator output against real process traces from Linux or Windows. Each extension would deepen the connection between the controlled test-case environment studied here and the full complexity of production operating system scheduling.

4. REFERENCES

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). Operating systems: three easy pieces. https://pages.cs.wisc.edu/~remzi/OSTEP/

Love, R. (2010). Linux kernel development (3rd ed.). Addison-Wesley.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Stallings, W. (2018). Operating systems: internals and design principles (9th ed.). Pearson.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

5. REPOSITORY LINK

https://github.com/chluraqt/os_cs.git

6. SOURCE CODE

```
# CPU Scheduling Simulator

import tkinter as tk
from tkinter import ttk, messagebox

BG          = "#fcfcff"
PANEL       = "#FFFFFF"
PANEL_ALT   = "#f1f1f1"
BORDER      = "#3a3a3d"
ACCENT      = "#2320f1"
ACCENT_DIM  = "#1c3ae4"

TEXT        = "#383842"
TEXT_MED    = "#383842"
TEXT_DIM    = "#383842"

BTN_DEL     = "#a32929"
BTN_DEL_HOV = "#a32929"
RUN_BG      = "#1c3ae4"
RUN_HOV     = "#2320f1"

IDLE_COL    = "#4a4a52"

FONT        = ("Segoe UI",  10)
FONT_SM     = ("Segoe UI",  9)
FONT_XS     = ("Segoe UI",  8)
FONT_BOLD   = ("Segoe UI",  10, "bold")
FONT_HEAD   = ("Segoe UI",  11, "bold")
FONT_TITLE  = ("Segoe UI",  13, "bold")
FONT_MONO   = ("Consolas",   9)

# Process colors — muted enough for dark
theme, distinct enough to read
PROC_COLORS = [
    "#4a90d9", "#3aaa8a", "#d4875a",
"#9b72d4",
    "#d46a8a", "#4aaabe", "#6ab870",
"#d4a03a",
    "#8a72c8", "#3aa0b0",
]

# SCHEDULING ALGORITHMS
# Each function receives a list of process
dicts and returns (results, gantt).

def attach_results(process, ct):
    """Compute and attach CT / TAT / WT to
a process dict in-place."""
    process["ct"]  = ct
    process["tat"] = ct - process["at"]
    process["wt"]  = process["tat"] -
process["bt"]

def fcfs(processes):
    """First-Come, First-Served — run
processes in arrival order."""
    procs = sorted(processes, key=lambda x:
(x["at"], x["pid"]))
    gantt, t = [], 0
    for p in procs:
        if t < p["at"]:
            gantt.append(("IDLE", t, p["at"]))
            t = p["at"]
        start = t
        t += p["bt"]
        attach_results(p, t)
        gantt.append((p["pid"], start, t))
    return procs, gantt


def sjf(processes):
    """Shortest Job First (non-preemptive) —
always pick the shortest available burst."""
    procs = [p.copy() for p in processes]
    gantt, t, done, rem = [], 0, [], procs[:]
    while rem:
        avail = [p for p in rem if p["at"] <= t]
        if not avail:
            nxt = min(p["at"] for p in rem)
            gantt.append(("IDLE", t, nxt))
            t = nxt
            continue
        sel = min(avail, key=lambda x:
(x["bt"], x["at"], x["pid"]))
        rem.remove(sel)
        start = t
        t += sel["bt"]
        attach_results(sel, t)
        gantt.append((sel["pid"], start, t))
        done.append(sel)
    return done, gantt


def srt(processes):
    """Shortest Remaining Time (preemptive
SJF) — tick-by-tick preemption."""
    procs = [p.copy() for p in processes]
    for p in procs:
        p["remaining"] = p["bt"]
    gantt, t, done = [], 0, []
    while len(done) < len(procs):
        avail = [p for p in procs if p["at"] <= t
and p["remaining"] > 0]
        if not avail:
            t += 1
            continue
```

```
        sel = min(avail, key=lambda x:
(x["remaining"], x["at"], x["pid"]))
        # Extend the last Gantt bar if the same
process continues
        if gantt and gantt[-1][0] == sel["pid"]:
            gantt[-1] = (gantt[-1][0], gantt[-1][1],
t + 1)
        else:
            gantt.append((sel["pid"], t, t + 1))
        sel["remaining"] -= 1
        t += 1
        if sel["remaining"] == 0:
            attach_results(sel, t)
            done.append(sel)
    return done, gantt


def round_robin(processes, tq):
    """Round Robin — rotate processes
through the queue using a fixed time
quantum."""
    procs = [p.copy() for p in processes]
    for p in procs:
        p["remaining"] = p["bt"]
    procs.sort(key=lambda x: x["at"])
    gantt, t, queue, done, idx = [], 0, [], [], 0

    # Seed the queue with processes that have
already arrived
    while idx < len(procs) and
procs[idx]["at"] <= t:
        queue.append(procs[idx])
        idx += 1

    while queue or idx < len(procs):
        if not queue:
            nxt = procs[idx]["at"]
            gantt.append(("IDLE", t, nxt))
            t = nxt
            while idx < len(procs) and
procs[idx]["at"] <= t:
                queue.append(procs[idx])
                idx += 1
        cur    = queue.pop(0)
        exec_t = min(tq, cur["remaining"])
        start  = t
        t     += exec_t
        cur["remaining"] -= exec_t
        # Admit newly arrived processes before
re-queuing the current one
        while idx < len(procs) and
procs[idx]["at"] <= t:
            queue.append(procs[idx])
            idx += 1
        gantt.append((cur["pid"], start, t))
        if cur["remaining"] == 0:
            attach_results(cur, t)
            done.append(cur)
        else:
            queue.append(cur)
    return done, gantt


def priority_nonpreemptive(processes):
    """Priority Scheduling (non-preemptive)
— lower number = higher priority."""
    procs = [p.copy() for p in processes]
    gantt, t, done, rem = [], 0, [], procs[:]
    while rem:
        avail = [p for p in rem if p["at"] <= t]
        if not avail:
            nxt = min(p["at"] for p in rem)
            gantt.append(("IDLE", t, nxt))
            t = nxt
            continue
        sel = min(avail, key=lambda x:
(x["priority"], x["at"], x["pid"]))
        rem.remove(sel)
        start = t
        t += sel["bt"]
        attach_results(sel, t)
        gantt.append((sel["pid"], start, t))
        done.append(sel)
    return done, gantt


def priority_round_robin(processes, tq):
    """Priority + Round Robin — run each
priority level as its own RR group."""
    procs  = [p.copy() for p in processes]
    levels = sorted(set(p["priority"] for p in
procs))
    gantt, done, t = [], [], 0

    for lv in levels:
        group = [p for p in procs if p["priority"]
== lv]
        for p in group:
            p["remaining"] = p["bt"]
        group.sort(key=lambda x: x["at"])
        queue, idx = [], 0
        t = max(t, group[0]["at"])
        while idx < len(group) and
group[idx]["at"] <= t:
            queue.append(group[idx])
            idx += 1
        while queue or idx < len(group):
            if not queue:
                t = group[idx]["at"]
                while idx < len(group) and
group[idx]["at"] <= t:
                    queue.append(group[idx])
                    idx += 1
            cur    = queue.pop(0)
```

```
        exec_t = min(tq, cur["remaining"])
        start  = t
        t     += exec_t
        cur["remaining"] -= exec_t
        while idx < len(group) and
group[idx]["at"] <= t:
            queue.append(group[idx])
            idx += 1
        gantt.append((cur["pid"], start, t))
        if cur["remaining"] == 0:
            attach_results(cur, t)
            done.append(cur)
        else:
            queue.append(cur)
    return done, gantt


# MAIN APPLICATION GUI

class CPUSchedulerApp(tk.Tk):

    def __init__(self):
        super().__init__()
        self.title("CPU Scheduler")
        self.configure(bg=BG)
        self.resizable(True, True)
        self.minsize(980, 660)
        self.geometry("1160x740")

        self.process_rows = []   # list of dicts,
one per process row
        self.row_counter  = 0   # monotonic ID
for PID labels

        self._setup_styles()
        self._build_ui()

        # Default demo processes
        for at, bt in [(0, 5), (2, 3), (4, 1)]:
            self._add_process_row(at=at, bt=bt)

        self._on_algo_change()

    # ── ttk widget styling
──────────────
```

```
        left.grid(row=0, column=0,
sticky="nsew", padx=(0, 12))
        right = tk.Frame(body, bg=BG)
        right.grid(row=0, column=1,
sticky="nsew")

        self._build_left(left)
        self._build_right(right)

    # ── LEFT PANEL — algorithm picker,
process list, run button
────────────────────────────────────────

    def _build_left(self, parent):
        """Build the control panel: algo
selector, time quantum, process table."""
        parent.rowconfigure(1, weight=1)
        parent.columnconfigure(0, weight=1)

        # ── Algorithm selection card
        algo_card = self._card(parent)
        algo_card.grid(row=0, column=0,
sticky="ew", pady=(0, 10))
        self._card_header(algo_card,
"Algorithm")

        tk.Label(algo_card, text="Scheduling
policy",
                 font=FONT_XS, bg=PANEL,
fg=TEXT_DIM).pack(
                 anchor="w", padx=14, pady=(8,
3))

        options = [
            ("First-Come, First-Served (FCFS)",
"fcfs"),
            ("Shortest Job First —
Non-Preemptive",    "sjf"),
            ("Shortest Remaining Time —
Preemptive",   "srt"),
            ("Round Robin (RR)",
"rr"),
            ("Priority Scheduling —
Non-Preemptive",   "priority"),
            ("Priority + Round Robin",
"prr"),
        ]
        self._algo_map = {o[0]: o[1] for o in
options}

        self.algo_var =
tk.StringVar(value=options[0][0])
        self.algo_combo = ttk.Combobox(
            algo_card,
textvariable=self.algo_var,
            values=[o[0] for o in options],
            state="readonly", font=FONT_SM,
width=40)
        self.algo_combo.current(0)
        self.algo_combo.pack(fill="x",
padx=14, pady=(0, 10))

self.algo_combo.bind("<<ComboboxSelecte
d>>",
                     lambda e:
self._on_algo_change())

        # Time quantum row —
packed/forgotten based on selected
algorithm
        self.tq_frame = tk.Frame(algo_card,
bg=PANEL)
        tk.Label(self.tq_frame, text="Time
quantum:",
                 font=FONT_SM, bg=PANEL,
fg=TEXT_MED).pack(side="left",
padx=(14, 8))
        self.tq_var = tk.IntVar(value=2)
        sb = tk.Spinbox(self.tq_frame,
from_=1, to=99,
                textvariable=self.tq_var,
                width=5,
font=FONT_MONO,
                relief="flat", bd=0,
                bg=PANEL_ALT, fg=TEXT,

buttonbackground=PANEL_ALT,
                insertbackground=ACCENT,
                highlightthickness=1,

highlightbackground=BORDER,
                highlightcolor=ACCENT)
        sb.pack(side="left")

        # ── Process list card
        proc_card = self._card(parent)
        proc_card.grid(row=1, column=0,
sticky="nsew", pady=(0, 10))
        proc_card.rowconfigure(3, weight=1)
        proc_card.columnconfigure(0,
weight=1)
        self._card_header(proc_card,
"Processes")

        tk.Label(proc_card,
                 text="Priority: lower number =
higher priority",
                 font=FONT_XS, bg=PANEL,
fg=TEXT_DIM).pack(
                 anchor="w", padx=14, pady=(6,
0))

        # Column header bar
```

```
        self.hdr_frame = tk.Frame(proc_card,
bg=PANEL_ALT)
        self.hdr_frame.pack(fill="x", padx=14,
pady=(8, 0))
        self._build_col_header()

        # Scrollable process rows canvas
        rows_outer = tk.Frame(proc_card,
bg=PANEL)
        rows_outer.pack(fill="both",
expand=True, padx=14, pady=(0, 4))

        self.rows_canvas =
tk.Canvas(rows_outer, bg=PANEL,
                                highlightthickness=0,
bd=0)
        vsb = ttk.Scrollbar(rows_outer,
orient="vertical",

command=self.rows_canvas.yview)

self.rows_canvas.configure(yscrollcommand
=vsb.set)
        vsb.pack(side="right", fill="y")
        self.rows_canvas.pack(side="left",
fill="both", expand=True)

        self.rows_frame =
tk.Frame(self.rows_canvas, bg=PANEL)
        self._rows_win =
self.rows_canvas.create_window(
            (0, 0), window=self.rows_frame,
anchor="nw")
        self.rows_frame.bind("<Configure>",
            lambda e:
self.rows_canvas.configure(

scrollregion=self.rows_canvas.bbox("all")))
        self.rows_canvas.bind("<Configure>",
            lambda e:
self.rows_canvas.itemconfig(
                self._rows_win, width=e.width))

        # Add / Reset buttons
        btn_bar = tk.Frame(proc_card,
bg=PANEL)
        btn_bar.pack(fill="x", padx=14,
pady=(2, 12))
        self._btn(btn_bar, "+ Add",
                  self._add_process_row,
PANEL_ALT, TEXT).pack(
                  side="left", padx=(0, 6))
        self._btn(btn_bar, "Reset",
                  self._clear_all, PANEL_ALT,
TEXT_DIM).pack(side="left")

        # ── Run simulation button —
full-width, visually dominant
        run_btn = tk.Button(parent,
                            text="▶  Run Simulation",
                            font=FONT_BOLD,
                            bg=RUN_BG,
fg="#111111",

activebackground=RUN_HOV,

activeforeground="#111111",
                            relief="flat", pady=12,
                            cursor="hand2",

command=self._run_simulation)
        run_btn.grid(row=2, column=0,
sticky="ew")

    # ── RIGHT PANEL — Gantt chart +
results table
────────────
```

```
        self.out_inner =
tk.Frame(self.out_canvas, bg=PANEL)
        self._out_win =
self.out_canvas.create_window(
            (0, 0), window=self.out_inner,
anchor="nw")
        self.out_inner.bind("<Configure>",
            lambda e: self.out_canvas.configure(

scrollregion=self.out_canvas.bbox("all")))
        self.out_canvas.bind("<Configure>",
            lambda e:
self.out_canvas.itemconfig(
                self._out_win, width=e.width))

        self._show_placeholder()

    # ── Shared widget factory helpers
──────────────────────────────────────
──────────────────────────────

    def _card(self, parent):
        """Return a dark-surface card frame
with a subtle border."""
        return tk.Frame(parent, bg=PANEL,
relief="flat",
                        highlightthickness=1,
highlightbackground=BORDER)

    def _card_header(self, parent, text):
        """Render a compact section label with
a left accent strip."""
        row = tk.Frame(parent, bg=PANEL)
        row.pack(fill="x", padx=14, pady=(12,
2))
        tk.Frame(row, bg=ACCENT,
width=3).pack(side="left", fill="y",
padx=(0, 8))
        tk.Label(row, text=text.upper(),
                 font=("Segoe UI", 8, "bold"),
                 bg=PANEL,
fg=ACCENT).pack(side="left",
anchor="w")

    def _btn(self, parent, text, cmd, bg, fg):
        """Factory for flat-style utility
buttons."""
        return tk.Button(parent, text=text,
font=FONT_SM,
                         bg=bg, fg=fg, relief="flat",
                         padx=12, pady=6,
cursor="hand2",

activebackground=BORDER,
                         activeforeground=TEXT,
                         command=cmd)

    # ── Column header for the process table
──────────────────────────────────────
──────────────────

    def _build_col_header(self):
        """Rebuild column headers — called
on load and whenever algo changes."""
        for w in
self.hdr_frame.winfo_children():
            w.destroy()
        cols = [("PID", 44), ("Arrival", 70),
("Burst", 62)]
        if self._needs_priority():
            cols.append(("Priority", 60))
        cols.append(("", 26))
        for txt, w in cols:
            tk.Label(self.hdr_frame, text=txt,
                     font=FONT_XS,
                     bg=PANEL_ALT,
fg=TEXT_DIM,
                     width=w // 7,
anchor="center").pack(
                     side="left", padx=1, pady=5)

    # ── Algorithm helper queries
──────────────────────────────────────
──────────────────────────────────

    def _algo_key(self):
        """Return the short key for the
currently selected algorithm."""
        return
self._algo_map.get(self.algo_combo.get(),
"fcfs")

    def _needs_priority(self):
        """True if the selected algorithm uses a
priority column."""
        return self._algo_key() in ("priority",
"prr")

    def _needs_quantum(self):
        """True if the selected algorithm needs
a time quantum input."""
        return self._algo_key() in ("rr", "prr")

    def _on_algo_change(self):
        """Show / hide quantum spinner and
priority column when algo changes."""
        self.tq_frame.pack_forget()
        if self._needs_quantum():
            self.tq_frame.pack(fill="x", pady=(0,
10))
        self._build_col_header()
        for row in self.process_rows:
            if self._needs_priority():
```

```
            row["pri_e"].pack(side="left",
padx=2)
        else:
            row["pri_e"].pack_forget()

    # ── Process row management
──────────────────────────────────────────────────────────────────────────────

    def _add_process_row(self, at="0",
bt=""):
        """Append a new editable process row
to the scrollable list."""
        self.row_counter += 1
        pid    = f"P{self.row_counter}"
        row_bg = PANEL_ALT if
self.row_counter % 2 == 0 else PANEL

        frame = tk.Frame(self.rows_frame,
bg=row_bg, pady=4)
        frame.pack(fill="x", padx=0, pady=1)

        pid_var = tk.StringVar(value=pid)
        at_var  = tk.StringVar(value=str(at))
        bt_var  = tk.StringVar(value=str(bt))
        pri_var = tk.StringVar(value="1")

        # PID badge
        tk.Label(frame, textvariable=pid_var,
                font=("Segoe UI", 8, "bold"),
                bg=ACCENT_DIM, fg=TEXT,
                width=4, anchor="center",
                pady=3,
padx=2).pack(side="left", padx=(6, 6))

        def _entry(var, w=7):
            """Return a styled entry widget
bound to the given StringVar."""
            return tk.Entry(frame,
textvariable=var,
                            font=FONT_MONO,
width=w,
                            relief="flat", bd=0,
                            fg=TEXT,
bg=PANEL_ALT if row_bg == PANEL else
PANEL,

insertbackground=ACCENT,
                            highlightthickness=1,

highlightbackground=BORDER,
                            highlightcolor=ACCENT)

        at_e  = _entry(at_var, 7)
        bt_e  = _entry(bt_var, 7)
        pri_e = _entry(pri_var, 5)
        at_e.pack(side="left", padx=3)
        bt_e.pack(side="left", padx=3)

        row_data = {
            "frame":   frame,
            "pid_var": pid_var,
            "at_var":  at_var,
            "bt_var":  bt_var,
            "pri_var": pri_var,
            "pri_e":   pri_e,
        }

        if self._needs_priority():
            pri_e.pack(side="left", padx=3)

        # Delete row button
        tk.Button(frame, text="✕",
                font=FONT_XS,
                bg=BTN_DEL, fg=TEXT,

activebackground=BTN_DEL_HOV,
                relief="flat", width=2, pady=3,
                cursor="hand2",
                command=lambda r=row_data:
self._remove_row(r)).pack(
                side="left", padx=(6, 4))

        self.process_rows.append(row_data)

    def _remove_row(self, row):
        """Remove a process row, enforcing a
minimum of one row."""
        if len(self.process_rows) <= 1:

messagebox.showwarning("Warning", "At
least one process is required.")
            return
        row["frame"].destroy()
        self.process_rows.remove(row)

    def _clear_all(self):
        """Destroy all rows and restore the
three default demo processes."""
        for r in self.process_rows:
            r["frame"].destroy()
        self.process_rows.clear()
        self.row_counter = 0
        for at, bt in [(0, 5), (2, 3), (4, 1)]:
            self._add_process_row(at=at, bt=bt)
        self._show_placeholder()

    # ── Output panel helpers
──────────────────────────────────────────────────────────────────────────────

    def _clear_output(self):
```

```
        """Remove all widgets from the output
inner frame."""
        for w in
self.out_inner.winfo_children():
            w.destroy()

    def _show_placeholder(self):
        """Render the idle state message before
any simulation has been run."""
        self._clear_output()
        tk.Label(self.out_inner,
                 text="Run a simulation to see
results here.",
                 font=FONT_SM, bg=PANEL,
fg=TEXT_DIM,
                 pady=48).pack()

    # ── Input validation + algorithm
dispatch
──────────────────────────────────────────────────────────────────────────────────────────────

    def _run_simulation(self):
        """Validate inputs, dispatch to the
correct algorithm, then display results."""
        if len(self.process_rows) < 3:
            messagebox.showerror("Error",
"Please add at least 3 processes.")
            return

        processes = []
        for row in self.process_rows:
            pid = row["pid_var"].get().strip()
            try:
                at = int(row["at_var"].get())
                bt = int(row["bt_var"].get())
                if bt < 1:
                    raise ValueError
            except ValueError:
                messagebox.showerror("Input
Error",
                    f"{pid}: Arrival Time and Burst
Time must be integers (BT ≥ 1).")
                return
            pri = 1
            if self._needs_priority():
                try:
                    pri = int(row["pri_var"].get())
                except ValueError:
                    messagebox.showerror("Input
Error",
                        f"{pid}: Priority must be an
integer.")
                    return
            processes.append({"pid": pid, "at":
at, "bt": bt, "priority": pri})
        key = self._algo_key()
        tq  = self.tq_var.get()

        dispatch = {
            "fcfs":     (fcfs,
"First-Come, First-Served (FCFS)",
[processes]),
            "sjf":      (sjf,                "Shortest
Job First — Non-Preemptive (SJF)",
[processes]),
            "srt":      (srt,                "Shortest
Remaining Time — Preemptive (SRT)",
[processes]),
            "rr":       (round_robin,
f"Round Robin (RR) · Quantum = {tq}",
[processes, tq]),
            "priority": (priority_nonpreemptive,
"Priority Scheduling — Non-Preemptive",
[processes]),
            "prr":      (priority_round_robin,
f"Priority + Round Robin · Quantum =
{tq}",   [processes, tq]),
        }
        fn, title, args = dispatch[key]
        result, gantt   = fn(*args)
        self._display_results(result, gantt, title,
key)

    # ── Results renderer
──────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────────────────────────
──────

    def _display_results(self, procs, gantt,
title, key):
        """Render the Gantt chart, process
table, and summary stats to the output
panel."""
        self._clear_output()

        # ── Results header
        hdr = tk.Frame(self.out_inner,
bg=PANEL_ALT)
        hdr.pack(fill="x")
        tk.Label(hdr, text=title,
                 font=FONT_BOLD,
bg=PANEL_ALT, fg=TEXT,
                 pady=10,
padx=16).pack(anchor="w")
        tk.Frame(self.out_inner, bg=BORDER,
height=1).pack(fill="x")

        # ── Build process→color mapping
for Gantt and legend
        pid_color = {}
        ci = 0
        for seg in gantt:
```

```
        p = seg[0]
        if p != "IDLE" and p not in
pid_color:
            pid_color[p] =
PROC_COLORS[ci %
len(PROC_COLORS)]
            ci += 1

    # ── Section label — Gantt Chart
    tk.Label(self.out_inner, text="GANTT
CHART",
             font=("Segoe UI", 8, "bold"),
             bg=PANEL,
fg=ACCENT).pack(anchor="w", padx=16,
pady=(12, 4))

    # Gantt canvas dimensions
    BAR_H     = 38
    TICK_H    = 20
    CANVAS_H  = BAR_H + TICK_H +
10
    MIN_SEG_W = 52

    seg_widths = [max(MIN_SEG_W,
(seg[2] - seg[1]) * 22) for seg in gantt]
    chart_w    = sum(seg_widths) + 8

    gantt_wrap = tk.Frame(self.out_inner,
bg=PANEL)
    gantt_wrap.pack(fill="x", padx=16,
pady=(0, 6))

    gc = tk.Canvas(gantt_wrap,
bg=PANEL, height=CANVAS_H,
                   highlightthickness=1,
highlightbackground=BORDER)
    h_sb = ttk.Scrollbar(gantt_wrap,
orient="horizontal",
                         command=gc.xview)

gc.configure(xscrollcommand=h_sb.set,
                 scrollregion=(0, 0, chart_w,
CANVAS_H))
    h_sb.pack(side="bottom", fill="x")
    gc.pack(side="top", fill="x")

    # Draw each segment bar
    x = 4
    for i, seg in enumerate(gantt):
        pid, s, e = seg
        w   = seg_widths[i]
        col = IDLE_COL if pid == "IDLE"
else pid_color[pid]

        gc.create_rectangle(x, 4, x + w, 4 +
BAR_H,
                            fill=col,
outline=PANEL, width=2)
        txt_col = TEXT_DIM if pid ==
"IDLE" else TEXT
        gc.create_text(x + w // 2, 4 +
BAR_H // 2,
                       text=pid, font=("Segoe UI",
8, "bold"),
                       fill=txt_col)
        # Tick mark at start
        tick_y = 4 + BAR_H
        gc.create_line(x, tick_y, x, tick_y +
5, fill=TEXT_DIM, width=1)
        gc.create_text(x + 2, tick_y + 13,
                       text=str(s),
font=FONT_XS,
                       fill=TEXT_DIM,
anchor="w")
        x += w

    # Final tick mark at end
    tick_y = 4 + BAR_H
    gc.create_line(x, tick_y, x, tick_y + 5,
fill=TEXT_DIM, width=1)
    gc.create_text(x + 2, tick_y + 13,
                   text=str(gantt[-1][2]),
                   font=FONT_XS,
fill=TEXT_DIM, anchor="w")

    # Color legend for process IDs
    leg = tk.Frame(self.out_inner,
bg=PANEL)
    leg.pack(anchor="w", padx=16,
pady=(2, 10))
    for pid, col in pid_color.items():
        dot = tk.Frame(leg, bg=col,
width=11, height=11)
        dot.pack(side="left")
        dot.pack_propagate(False)
        tk.Label(leg, text=pid,
font=FONT_XS,
                 bg=PANEL,
fg=TEXT_MED).pack(side="left", padx=(3,
10))

    tk.Frame(self.out_inner, bg=BORDER,
height=1).pack(fill="x", padx=16, pady=(0,
6))

    # ── Section label — Process Table
    tk.Label(self.out_inner,
text="PROCESS TABLE",
             font=("Segoe UI", 8, "bold"),
             bg=PANEL,
fg=ACCENT).pack(anchor="w", padx=16,
pady=(4, 6))
```

```
        has_pri = key in ("priority", "prr")
        cols    = ["PID", "Arrival", "Burst"]
        if has_pri:
            cols.append("Priority")
        cols += ["Completion", "Turnaround",
"Waiting"]

        # Table rendered as a bordered grid
using 1-px BORDER gaps
        tbl = tk.Frame(self.out_inner,
bg=BORDER)
        tbl.pack(fill="x", padx=16, pady=(0,
12))
        for j in range(len(cols)):
            tbl.columnconfigure(j, weight=1)

        col_w = 9

        # Header row
        for j, c in enumerate(cols):
            cell = tk.Frame(tbl,
bg=PANEL_ALT)
            cell.grid(row=0, column=j, padx=1,
pady=1, sticky="nsew")
            tk.Label(cell, text=c,
                     font=FONT_XS,
                     bg=PANEL_ALT,
fg=TEXT_MED,
                     width=col_w,
anchor="center").pack(padx=6, pady=7)

        # Data rows — alternate PANEL /
PANEL_ALT for readability
        for i, p in enumerate(procs):
            row_bg = PANEL if i % 2 == 0 else
PANEL_ALT
            vals   = [p["pid"], p["at"], p["bt"]]
            if has_pri:
                vals.append(p["priority"])
            vals += [p["ct"], p["tat"], p["wt"]]
            for j, v in enumerate(vals):
                cell = tk.Frame(tbl, bg=row_bg)
                cell.grid(row=i + 1, column=j,
padx=1, pady=1, sticky="nsew")
                tk.Label(cell, text=str(v),
                         font=FONT_MONO,
bg=row_bg, fg=TEXT,
                         width=col_w,
anchor="center").pack(padx=6, pady=6)

        tk.Frame(self.out_inner, bg=BORDER,
height=1).pack(fill="x", padx=16, pady=(0,
6))

        # ── Summary stat cards — Average
WT and Average TAT
        n      = len(procs)
        avg_wt  = sum(p["wt"]  for p in procs) /
n
        avg_tat = sum(p["tat"] for p in procs) /
n

        avg_row = tk.Frame(self.out_inner,
bg=PANEL)
        avg_row.pack(fill="x", padx=16,
pady=(4, 18))

        for label, val, fg in [
            ("Avg. Waiting Time",      avg_wt,
ACCENT),
            ("Avg. Turnaround Time",   avg_tat,
"#6abf77"),
        ]:
            card = tk.Frame(avg_row,
bg=PANEL_ALT,
                            highlightthickness=1,
highlightbackground=BORDER)
            card.pack(side="left", expand=True,
fill="both", padx=(0, 8))
            tk.Label(card, text=label.upper(),
                     font=("Segoe UI", 8),
                     bg=PANEL_ALT,
fg=TEXT_DIM).pack(pady=(12, 2))
            tk.Label(card, text=f"{val:.2f}",
                     font=("Segoe UI", 24, "bold"),
                     bg=PANEL_ALT,
fg=fg).pack(pady=(0, 12))

        self.out_canvas.yview_moveto(0)


# SIMULATION POINT

if __name__ == "__main__":
    app = CPUSchedulerApp()
    app.mainloop()
```